

Pinakes on Windows Server — Operations Runbook

Who this is for: whoever has administrator access to the Windows Server that runs `testpinakes.wacclearinghouse.org` . No prior knowledge of the application is assumed.

What this covers: the two things you do after the install is finished.

- **Part A** — bring the deployment to a healthy, populated state. Do this once.
- **Part B** — deploy a new version from GitHub. Do this whenever there is a release.

For a first install on a new machine, use [WINDOWS-SERVER-INSTALL.md](#) instead.

Everything here is scripted. The scripts live in the repository under `deploy/windows\` , so they arrive on the server with the code itself.

Conventions

- Every command runs in an **elevated PowerShell** window: right-click Start, then *Terminal (Admin)* or *Windows PowerShell (Admin)*.
- Paths assume the default layout from the install guide:

Path	Contents
<code>C:\Pinakes\app</code>	the git clone of the repository
<code>C:\Pinakes\venv</code>	the Python virtual environment
<code>C:\Pinakes\data\articles.db</code>	the SQLite database — the only real state
<code>C:\Pinakes\logs</code>	service and job logs
<code>C:\Pinakes\backups</code>	database backups (created by the scripts)

If this server installed somewhere else, set the root once and every script follows it:

```
[Environment]::SetEnvironmentVariable("PINAKES_ROOT", "D:\Apps\Pinakes", "Machine")
```

Then close and reopen PowerShell.

- The application is one Windows service, `Pinakes-web` , running the Flask app behind waitress on `127.0.0.1:8080` . IIS reverse-proxies the public hostname to it.
-

Part A — Bring the deployment to healthy

As of 2026-08-27 the test deployment has three problems. Working through this part fixes all three. Each step says what it is for, so you can skip any that have already been done.

What is actually wrong (read this first)

1. **The server is running old code that cannot fetch articles.** `/health` reports commit `e0f82ec` ; `main` is four commits ahead. The missing fix is commit `138f4cd` . CrossRef stopped accepting a sort order combined with cursor paging and now returns `400 sort-criteria-incompatible-with-cursor` for the exact request the deployed code sends. Every deep page fails, so the index stays empty. This is why `/api/articles` returns `{"articles":[],"total":0}` .
2. **IIS/ARR is caching API responses for an hour, including empty ones.** The app was sending `Cache-Control: public, max-age=3600` , which ARR honours by storing its own copy — so requests never reached the app at all. During the initial seed that means the site keeps serving empty JSON out of the proxy cache while the database behind it fills up, and the site looks broken when it is not. Fixed in the app by sending `private` instead; deploying step A4 is what applies it.
3. **Nothing is scheduled.** The install guide created a `Pinakes-Scheduler` service pointed at `scheduler.py` , but that file was deleted from the repository in commit `fa25cae` . That service cannot start. The equivalent jobs on the Fly deployment run from GitHub Actions against `pinakes.xyz` , which does nothing for this server. So there is currently no daily fetch, no backup, and no maintenance. Step A6 registers them as scheduled tasks.

A1. Get the deployment tooling onto the server

The update scripts ship inside the repository, so the very first update is the only one you do by hand. Everything after this uses the script.

```
Set-Location C:\Pinakes\app
git fetch origin
git log --oneline HEAD..origin/main
```

That last command lists what is about to be deployed. Expect four commits, ending with `138f4cd fix(fetch): drop sort+cursor combo rejected by CrossRef` .

If `git status` shows local modifications, stop and send them to Justin before continuing — the next step discards them.

```
git reset --hard origin/main
```

You now have `C:\Pinakes\app\deploy\windows\` with six scripts in it.

A2. Check prerequisites

```
& C:\Pinakes\app\deploy\windows\Test-PinakesHealth.ps1
```

This prints PASS / FAIL / WARN for the service, the app, the database, the public URL, and whether the code matches GitHub. Nothing is changed.

Two results are expected to be unhappy at this point and will be fixed below: `Index contains articles` (0 articles) and `Service running checked-out code` (the tree moved in A1 but the service has not restarted yet).

If it reports that the service `Pinakes-Web` does not exist, or that the venv Python is missing, this machine is not installed the way the guide assumes — go back to [WINDOWS-SERVER-INSTALL.md](#).

A3. Confirm the admin token

The fetch and maintenance endpoints require a bearer token. The service already has one (`/health` reports `"admin_auth": "configured"`), but the scheduled tasks need to read the same value from the machine environment:

```
[Environment]::GetEnvironmentVariable("PINAKES_ADMIN_TOKEN", "Machine")
```

If that prints nothing, the token was set another way (for example directly on the NSSM service). Set it at Machine scope so the tasks can find it, using the **same value the service is running with**:

```
[Environment]::SetEnvironmentVariable("PINAKES_ADMIN_TOKEN", "<token>", "Machine")  
Restart-Service Pinakes-Web
```

Justin has the token. If nobody has it, generate a new one and set it — nothing depends on the old value:

```
$token = [Convert]::ToBase64String([Security.Cryptography.RandomNumberGenerator]::GetBytes(32))  
[Environment]::SetEnvironmentVariable("PINAKES_ADMIN_TOKEN", $token, "Machine")  
Write-Host "New token (send to Justin over a secure channel): $token"  
Restart-Service Pinakes-Web
```

A4. Deploy the current code

```
& C:\Pinakes\app\deploy\windows\Update-Pinakes.ps1
```

The tree is already at `origin/main` from A1, so this mostly restarts the service onto it — but run it anyway rather than restarting by hand, because it takes a backup, verifies the result, and rolls back if the new code will not start.

Expect roughly two minutes. The last lines should read:

```
[...] == Update complete  
[...] OK   Version:    138f4cd  
[...]     Admin auth: configured  
[...]     Database:   reachable  
[...] OK   Public URL: https://testpinakes.wacclearinghouse.org returned HTTP 200
```

If it fails, it rolls back on its own and tells you where the transcript is. See [When an update fails](#).

A5. Remove the dead scheduler service

Only if one exists on this machine:

```
Get-Service Pinakes-Scheduler -ErrorAction SilentlyContinue
```

If that returns something, it is the broken service described above. Remove it:

```
Stop-Service Pinakes-Scheduler -ErrorAction SilentlyContinue  
nssm remove Pinakes-Scheduler confirm
```

A6. Register the scheduled jobs

```
& C:\Pinakes\app\deploy\windows\Install-PinakesTasks.ps1 -IncludeUpdateCheck
```

This creates five scheduled tasks, all running as SYSTEM:

Task	When	What
Pinakes-NightlyBackup	02:00 daily	Local database backup to C:\Pinakes\backups
Pinakes-DailyFetch	03:23 daily	Incremental CrossRef/RSS fetch
Pinakes-Prewarm	04:15 daily	Rebuild cached aggregates
Pinakes-WeeklyMaintenance	04:45 Sundays	Weekly maintenance pass
Pinakes-UpdateCheck	08:00 daily	Reports if the server falls behind GitHub. Reports only — never deploys.

Backup runs before the fetch, so there is always a copy from before the day's writes. Prewarm runs after, so it warms new data rather than old.

Confirm:

```
& C:\Pinakes\app\deploy\windows\Install-PinakesTasks.ps1 -List
```

The weekly saved-search email digest is **not** registered by default. Add `-IncludeDigests` only once `PINAKES_ALERTS_ENABLED=1` and the sending domain is verified in Resend; until then the endpoint is a no-op.

A7. Run the initial fetch

This is the long one. It walks every indexed journal's full CrossRef history and writes a few hundred MB.

```
& C:\Pinakes\app\deploy\windows\Invoke-PinakesAdmin.ps1 -Endpoint Fetch
```

It returns immediately with `{"status":"fetch started"}` — the work happens in a background thread inside the web service. Watch it:

```
Get-Content C:\Pinakes\logs\web-stdout.log -Wait -Tail 30
```

You will see a line per journal, like `[College English] +412 articles`. Expect **30 to 90 minutes** for the full corpus. Press Ctrl+C to stop watching; that does not stop the fetch.

If you see `400` and `sort-criteria-incompatible-with-cursor`, the server is still on the old code — A4 did not take. Check `/health` and go back to A1.

A8. Verify

Once the log has gone quiet:

```
& C:\Pinakes\app\deploy\windows\Test-PinakesHealth.ps1
```

Everything should now be PASS. In particular:

- `Index contains articles` — a real count, not 0.

- `API responses not shared-cacheable` — `Cache-Control: private, max-age=3600` . If this still says `public` , the deploy did not land.
- `Public and local versions agree` — the proxy is not serving a stale copy.

Then look at the site: <https://testpinakes.wacclearinghouse.org/explore> — pick any tool from the accordion and confirm a chart draws.

If pages look stale or empty right after the deploy, IIS/ARR may still be holding copies cached from before the fix. They expire within an hour on their own. To clear them immediately, recycle the ARR cache from IIS Manager (server node → *Application Request Routing Cache* → *Clear all cached content*), or `iisreset` if that is acceptable on this box.

A9. Tell Justin

Send him the output of:

```
& C:\Pinakes\app\deploy\windows\Test-PinakesHealth.ps1
```

He can also check from his own machine at any time, without server access — see [What Justin can do without server access](#).

Part B — Push an update from GitHub

This is the routine after Part A. It takes about two minutes.

B0. Before you start (Justin's side)

Nothing deploys to this server automatically. A change reaches it only when someone runs the update script. The change must be merged to `main` on <https://github.com/justalewis/Rhet-Comp-Index> first; the repository's own CI runs the test suite on every push to `main` , so a red build is visible there before anyone deploys.

B1. See what would change

```
& C:\Pinakes\app\deploy\windows\Update-Pinakes.ps1 -Check
```

Reports the deployed commit, the target commit, and every commit in between. **Changes nothing** — safe to run any time, and it does not need an elevated shell.

If it says `Already up to date` , you are done.

B2. Deploy

```
& C:\Pinakes\app\deploy\windows\Update-Pinakes.ps1
```

In order, the script:

1. Fetches from GitHub and compares commits. Exits early if there is nothing to do.
2. Takes a hot backup of the database to `C:\Pinakes\backups` .

3. Stops `Pinakes-Web` .
4. Moves the working tree to the target commit.
5. Reinstalls Python dependencies.
6. Smoke-tests that the new code imports, against a scratch database.
7. Starts the service and polls `/health` until it reports the **new** commit.
8. Rolls back automatically if any of 4–7 fails.

Step 7 is the point of the exercise. The app derives its version from `git rev-parse --short HEAD` when the process starts, so `/health` reporting the new SHA is proof the service actually restarted onto the new code — not merely that it is still running.

B3. Verify

```
& C:\Pinakes\app\deploy\windows\Test-PinakesHealth.ps1
```

All PASS. Then load the site and click through one tool.

B4. Useful variations

Deploy a specific commit or tag, rather than the tip of `main` :

```
& .\Update-Pinakes.ps1 -Ref v1.4.0
```

Deliberately step **back** to a known-good commit — this is the manual rollback, and it takes a backup and verifies just like a forward deploy:

```
& .\Update-Pinakes.ps1 -Ref e0f82ec
```

Someone edited a file directly on the server and the script refuses to discard it. Review the diff first (`git diff`), send it to Justin so it can be committed properly, then:

```
& .\Update-Pinakes.ps1 -Force
```

Slow server, or a very large database — startup runs schema migrations before it answers, so give it longer:

```
& .\Update-Pinakes.ps1 -TimeoutSec 600
```

When an update fails

The script rolls back on its own and prints the last 40 lines of the service error log. The site should be back on the previous version before it exits. To confirm:

```
& C:\Pinakes\app\deploy\windows\Test-PinakesHealth.ps1  
Invoke-RestMethod http://127.0.0.1:8080/health
```

A full transcript of every run is written to `C:\Pinakes\logs\update-<timestamp>.log` . Send that to Justin along with:

```
Get-Content C:\Pinakes\logs\web-stderr.log -Tail 60
```

If rollback also failed — the script says so explicitly, and the site is down. Recover by hand:

```
Set-Location C:\Pinakes\app
git log --oneline -10          # find the last known-good commit
git reset --hard <that-commit>
C:\Pinakes\venv\Scripts\python.exe -m pip install -r requirements.txt
Start-Service Pinakes-Web
Invoke-RestMethod http://127.0.0.1:8080/health
```

If a schema migration corrupted the database — restore the backup the script took before the deploy:

```
Stop-Service Pinakes-Web
Get-ChildItem C:\Pinakes\backups | Sort-Object LastWriteTime -Descending | Select-Object -First
5
Copy-Item C:\Pinakes\backups\articles-<timestamp>-pre-<sha>.db C:\Pinakes\data\articles.db
-Force
Remove-Item C:\Pinakes\data\articles.db-wal, C:\Pinakes\data\articles.db-shm -ErrorAction
SilentlyContinue
Start-Service Pinakes-Web
```

Restore the code to the matching version too, or the migration will simply run again.

Reference

The scripts

All in `C:\Pinakes\app\deploy\windows\`. Each supports `Get-Help <script> -Full`.

Script	Purpose
<code>Update-Pinakes.ps1</code>	Deploy from GitHub, with backup, verification, and rollback. Part B.
<code>Test-PinakesHealth.ps1</code>	Full health check. Add <code>-Remote</code> to check over HTTPS from anywhere.
<code>Invoke-PinakesAdmin.ps1</code>	Trigger a fetch, prewarm, maintenance, or digest run.
<code>Backup-Pinakes.ps1</code>	Take a database backup on demand.
<code>Install-PinakesTasks.ps1</code>	Register, list, or remove the scheduled jobs.
<code>Pinakes.Common.ps1</code>	Shared config and helpers. Dot-sourced by the others; not run directly.
<code>sqlite_backup.py</code>	Hot-copy helper used by the backup path.

Everyday commands

```
# Health
& C:\Pinakes\app\deploy\windows\Test-PinakesHealth.ps1

# What version is live right now?
Invoke-RestMethod http://127.0.0.1:8080/health

# Has this server fallen behind GitHub?
& C:\Pinakes\app\deploy\windows\Update-Pinakes.ps1 -Check

# Fetch new articles now, without waiting for 03:23
& C:\Pinakes\app\deploy\windows\Invoke-PinakesAdmin.ps1 -Endpoint Fetch

# Back up before doing something risky
& C:\Pinakes\app\deploy\windows\Backup-Pinakes.ps1 -Label before-<whatever>

# Restart the app
Restart-Service Pinakes-Web

# Job history
Get-Content C:\Pinakes\logs\admin-tasks.log -Tail 40

# Live application log
Get-Content C:\Pinakes\logs\web-stdout.log -Wait -Tail 30

# Scheduled task status
& C:\Pinakes\app\deploy\windows\Install-PinakesTasks.ps1 -List
```

Logs

File	Contents
C:\Pinakes\logs\web-stdout.log	Application log — fetch progress, request errors
C:\Pinakes\logs\web-stderr.log	Startup failures, tracebacks, migration errors
C:\Pinakes\logs\admin-tasks.log	One line per scheduled job run
C:\Pinakes\logs\update-<timestamp>.log	Full transcript of each deploy

What Justin can do without server access

Given the admin token, from any machine:

```
# Health and version, from outside
Invoke-RestMethod https://testpinakes.wacclearinghouse.org/health

# Full external health check (clone the repo, then)
& .\deploy\windows\Test-PinakesHealth.ps1 -Remote

# Trigger a fetch remotely
$H = @{ Authorization = "Bearer <token>" }
Invoke-RestMethod -Uri https://testpinakes.wacclearinghouse.org/fetch -Method POST -Headers $H
```

A 409 from that last one means a fetch is already running — that is the guard against two writers on one SQLite file, not an error.

What still needs someone with server access: deploying code, restarting the service, reading logs, and anything to do with IIS.

Troubleshooting

`/health` reports an old version after an update. The working tree moved but the service did not restart. `Restart-Service Pinakes-Web`. The update script checks for exactly this and will not report success without it.

The site shows stale or empty data, but `Test-PinakesHealth.ps1` passes. IIS/ARR is serving from its cache. The health script cache-busts every request, which is why it disagrees with a browser. Clear the ARR cache in IIS Manager, or wait out the hour. If the check reports `Cache-Control: public`, the server is running a build older than this fix — deploy.

403 from `Invoke-PinakesAdmin.ps1`. The machine-scope `PINAKES_ADMIN_TOKEN` does not match what the service is running with. Set them to the same value and `Restart-Service Pinakes-Web`.

503 from `Invoke-PinakesAdmin.ps1`. The service has no token in its environment at all. See A3.

A scheduled task shows a non-zero last result. Read `C:\Pinakes\logs\admin-tasks.log` for the reason, then run the same command by hand to see it fail in front of you.

`database is locked`. Confirm `DB_PATH` is on a local NTFS volume and not a network share, and that no leftover `Pinakes-Scheduler` process is running as a second writer. More in [WINDOWS-SERVER-INSTALL.md](#).

502 Bad Gateway from IIS, but `http://127.0.0.1:8080` works. The app is fine; the reverse proxy is not. See the ARR section of the install guide.

Runbook version 2026-08-27. Companion to [WINDOWS-SERVER-INSTALL.md](#). Scripts in `deploy/windows/`.